# Reescrita de consultas SQL com LLMs: comparação entre modelos locais em CPU e um modelo pago

**Luiz Eduardo Treméa**[1], **Fahad Kalil**[1]

[1]Ciência da Computação – Atitus Educação
Passo Fundo – RS – Brasil

1123471@atitus.edu.br, fahad.kalil@atitus.edu.br

***Abstract.*** *This work evaluates SQL query rewriting by large language models under a method in which the model only proposes a rewrite and a validator classifies it by running it on PostgreSQL, comparing the results and checking the estimated cost. Two local models running on CPU, llama3.2 (3B) and llama3 (8B), were compared with GPT-4o on four TPC-H queries and on 18 inefficient queries over a controlled database, under four context phases. No model optimized a TPC-H query. On the controlled database, 15% to 19% of each model's runs returned a different result and were rejected; GPT-4o optimized 58% of the runs, llama3 (8B) 38% and llama3.2 (3B) 33%, and no context phase brought a local model to the level of GPT-4o. Comparing results on a single instance does not prove equivalence: for one query, 32 of the 36 accepted runs are equivalent only on this database instance.*

***Resumo.*** *Este trabalho avalia a reescrita de consultas SQL por modelos de linguagem de grande escala (LLMs) sob uma metodologia em que o modelo apenas propõe a reescrita e um validador a classifica, executando-a no PostgreSQL, comparando os resultados e verificando o custo estimado. Dois modelos locais executados em CPU, o llama3.2 (3B) e o llama3 (8B), foram comparados ao GPT-4o em quatro consultas do TPC-H e em 18 consultas ineficientes de uma base controlada, em quatro fases de contexto. No TPC-H, nenhum modelo otimizou consultas. Na base controlada, de 15% a 19% das execuções de cada modelo alteraram o resultado e foram barradas; o GPT-4o otimizou 58% das execuções, o llama3 (8B), 38%, e o llama3.2 (3B), 33%; nenhuma fase de contexto levou um modelo local ao nível do GPT-4o. A comparação sobre uma única instância não prova equivalência: em uma das consultas, 32 das 36 execuções aceitas só são equivalentes nesta base.*

## 1. Introdução

Em sistemas de informação baseados em bancos de dados relacionais, o tempo de execução das consultas afeta diretamente o custo de infraestrutura e o tempo de resposta das aplicações. Otimizações são realizadas por um componente do Sistema Gerenciador de Banco de Dados (SGBD), chamado de *planner* (planejador) ou *optimizer* (otimizador), que escolhe um plano de execução por estimativas de custo, mas só considera as transformações que conhece (Chaudhuri, 1998). Uma consulta mal formulada com subconsulta correlacionada, função sobre coluna indexada ou operação redundante pode ficar fora desse repertório e reescrevê-la de forma equivalente produz, então, ganhos que o otimizador sozinho, eventualmente, não alcança.

Trabalhos recentes usam modelos de linguagem de grande escala (LLMs) para sugerir essas reescritas, seja escolhendo regras de um catálogo (Li; Yuan *et al.*, 2024; Sun *et al.*, 2025), seja gerando diretamente a consulta reescrita (Liu; Mozafari, 2025; Dharwada *et al.*, 2026). A geração direta pode produzir consultas sintaticamente válidas e semanticamente incorretas (Huang; Yu *et al.*, 2024) e, por consequência, cada reescrita precisa ser verificada antes de ser aceita. Os trabalhos citados variam entre modelos pagos, abertos de grande porte ou ajustados para a tarefa (Seção 3). Dessa forma, medir quanto entrega um modelo pequeno, executado localmente e sem ajuste fino ou treinamento específico, é uma opção a ser investigada visando usuários que não possam enviar consultas e esquema a um serviço externo e não disponham de GPU.

Duas perguntas orientam o estudo. A primeira é em que medida as reescritas propostas por modelos de linguagem reduzem o custo das consultas e quantas devolvem linhas diferentes das da consulta original, quando cada reescrita é executada no próprio banco de dados e comparada à original. A segunda é se um modelo local, sem custo por chamada e sem envio de dados a terceiros, alcança o modelo pago ao receber progressivamente mais contexto sobre a base de dados investigada. Alcançar, aqui, significa otimizar, em alguma fase, pelo menos tantas execuções quanto o modelo pago, sob a mesma validação.

O objetivo geral é comparar modelos locais e um modelo pago na reescrita de consultas SQL, sob uma metodologia em que cada reescrita é validada no próprio SGBD. Para isso, o trabalho reúne a metodologia de avaliação a partir das técnicas descritas na literatura, constrói um protótipo em que o modelo recebe apenas texto e nunca acessa o banco, classifica cada reescrita em cinco categorias e executa a comparação em dois cenários, um subconjunto do TPC-H e uma base controlada, em quatro fases de contexto e três repetições.

## 2. Referencial teórico

### 2.1. Otimização de consultas e uso de inteligência artificial (IA)

A otimização baseada em custo remonta ao System R, que estimava a seletividade dos predicados e escolhia, por programação dinâmica, a ordem de junção mais barata (Selinger *et al.*, 1979). Chaudhuri (1998) descreve o otimizador em três blocos: transformações que preservam o resultado, um modelo que estima o custo de cada alternativa e um algoritmo de busca no espaço de planos. O ponto fraco desse arranjo é a estimação. Ao avaliar otimizadores sobre o *Join Order Benchmark* (JOB), Leis *et al.* (2015) mostraram que os erros de estimativa de cardinalidade degradam a qualidade dos planos mais do que o modelo de custo ou o algoritmo de enumeração. O paradigma de *Learned Query Optimization* (LQO) leva o aprendizado de máquina para dentro do otimizador, com sistemas que aprendem a escolher planos, como o *Neo* e o *Bao* (Marcus; Negi; Mao; Zhang *et al.*, 2019; Marcus; Negi; Mao; Tatbul *et al.*, 2021), ou estimadores de cardinalidade, como o *NeuroCard* e o *FLAT* (Yang *et al.*, 2021; Zhu *et al.*, 2021).

Reescrever uma consulta SQL significa transformá-la em outra que produza o mesmo resultado, porém de forma mais eficiente. A técnica antecede os modelos de linguagem: Starburst (Pirahesh; Hellerstein; Hasan, 1992) aplicava regras de reescrita sobre a consulta antes do planejamento físico e o Apache Calcite (Begoli *et al.*, 2018) reúne

uma biblioteca de regras de transformação testadas e reaproveitáveis. O limite dessas abordagens é estrutural, porque elas só aplicam transformações codificadas previamente.

Sistemas baseados em LLMs, apresentados na Seção 3, seguem duas abordagens. Na primeira, o modelo escolhe e ordena regras de um catálogo existente, como em *LLM-R2* e em *R-Bot* (Li; Yuan *et al.*, 2024; Sun *et al.*, 2025). Na segunda, o modelo propõe a própria consulta reescrita, como em *QUITE*, em *LITHE* e *GenRewrite*, que expressa em linguagem natural as transformações que aprende (Song *et al.*, 2025; Dharwada *et al.*, 2026; Liu; Mozafari, 2025). Esses sistemas se apoiam no aprendizado em contexto, no qual o modelo resolve tarefas novas a partir do que recebe via *prompt* (Brown *et al.*, 2020) e em técnicas como o raciocínio passo a passo, a autoconsistência (Wei *et al.*, 2022; Wang *et al.*, 2023) e a geração aumentada por recuperação (*Retrieval-Augmented Generation* - RAG) (Lewis *et al.*, 2020).

A reescrita não se confunde com o *Text-to-SQL*, que traduz uma pergunta em linguagem natural para uma consulta, mas essa tarefa vizinha mostra que a saída de um LLM não pode ser aceita sem verificação: em *BIRD* (Li; Hui *et al.*, 2023), o melhor modelo avaliado, o ChatGPT, alcançou 40,08% de acurácia de execução, contra 92,96% das pessoas, e erros frequentes aparecem também em *Spider* (Yu *et al.*, 2018). Em tarefas de banco de dados (Li; Zhou; Zhao, 2024), os LLMs estão sujeitos à alucinação de IA (*AI hallucinations*), isto é, à produção de saídas plausíveis, porém incorretas (Huang; Yu *et al.*, 2024). Assim, cada reescrita precisa de uma instância externa que a valide.

### 2.2. Síntese crítica da literatura

A literatura segue, portanto, dois caminhos: levar o aprendizado de máquina para dentro do otimizador ou mantê-lo intacto e reescrever as consultas externamente, com o apoio de modelos de linguagem. Em ambos, decidir se duas consultas são equivalentes é indecidível no caso geral (Abiteboul; Hull; Vianu, 1995), e os provadores automáticos cobrem apenas fragmentos do SQL ou domínios finitos (Chu *et al.*, 2017; Zhou; Arulraj; Navathe; Harris; Xu, 2019; Zhou; Arulraj; Navathe; Harris; Wu, 2022; He *et al.*, 2024). Por isso, a validação deste trabalho recorre à comparação dos resultados produzidos sobre um conjunto concreto de dados: resultados diferentes refutam a equivalência, mas resultados iguais sobre uma instância não a comprovam. É essa a prática de parte dos trabalhos sobre sistemas de reescrita por LLM, como: ReSequel, SPA e a etapa por amostras do LITHE (Fathollahzadeh; Mansour; Boehm, 2026; Huang; Miao, 2026; Dharwada *et al.*, 2026).

## 3. Trabalhos relacionados

Os estudos a seguir aplicam modelos de linguagem à reescrita de consultas SQL. Cada item termina com o que este estudo aproveitou do trabalho ou em que difere dele.

- **IA na otimização de consultas** (Amaral; Andrade; Campos, 2025). Avalia o ChatGPT na reescrita de consultas em PostgreSQL, com *scripts* em Python que automatizam a execução e a coleta de métricas; em cenários com muitos *joins*, a reescrita reduziu o tempo de execução em até 90%. Como este estudo, avalia um modelo existente em vez de propor um sistema, mas com um único modelo pago; aqui, dois modelos locais entram na comparação.
- **LLM-R2** (Li; Yuan *et al.*, 2024). O modelo escolhe sequências de regras do Apache Calcite, com uma demonstração selecionada por um modelo contrastivo,

e a reescrita é equivalente por construção. Este estudo usa a mesma verificação de que a reescrita executa no próprio banco (Seção 4.6), mas aqui o modelo devolve o SQL reescrito, cuja equivalência precisa ser verificada por execução.

- **R-Bot** (Sun *et al.*, 2025). Uma arquitetura RAG recupera receitas de reescrita e evidências de manuais e fóruns, e o modelo aplica as regras passo a passo, com autorreflexão. Este estudo adota o mesmo modelo, o GPT-4o, e a mesma versão do SGBD, o PostgreSQL 14 (Seções 4.2 e 4.3), mas o modelo não recebe evidências externas, apenas o contexto extraído do banco.
- **LITHE** (Dharwada *et al.*, 2026). Reescreve no espaço da consulta (*SQL-to-SQL*), combinando regras sensíveis ao banco com uma busca em árvore de Monte Carlo guiada pelas probabilidades de *tokens* do modelo. Do LITHE vêm duas práticas adotadas aqui, o custo estimado como medida principal de ganho e o esquema com o tamanho das tabelas no *prompt*, versão reduzida das estatísticas e seletividades usadas no LITHE (Seção 4); não há, porém, busca sobre alternativas: cada execução deste estudo é uma única chamada ao modelo.
- **QUITE** (Song *et al.*, 2025). Coordena agentes por uma máquina de estados finita, com retorno do banco via comando `EXPLAIN` e *hints* que fixam o plano, o que lhe permite ir além das regras fixas. Aqui, o plano do `EXPLAIN` entra como contexto da fase C3 (Seção 4.4), mas é entregue uma única vez, como texto, sem retorno sobre as reescritas propostas.
- **ReSequel** (Fathollahzadeh; Mansour; Boehm, 2026). Agrupa a carga em *templates*, gera reescritas com LLMs, verifica-as por execução sobre amostras dos dados e as reaproveita por *template*. A classificação de consultas mal escritas proposta no trabalho serviu de base para as categorias da base controlada deste estudo.
- **SPA** (Huang; Miao, 2026). Ajusta um modelo local por aprendizado por reforço, com o plano de execução e o tempo medido como recompensa. É o trabalho mais próximo do cenário de modelo local deste estudo, com a diferença de que aqui os modelos locais são avaliados sem ajuste fino.

Este trabalho não propõe um sistema concorrente aos descritos. A metodologia (Seção 4) combina práticas desses trabalhos, com foco em comparar, sob a mesma sequência de testes e a mesma validação, dois modelos locais pequenos, sem ajuste fino e executados em CPU, usando como *baseline* o GPT-4o, modelo principal do R-Bot e do LITHE. A Tabela 1 compara os sete sistemas de reescrita, incluindo o GenRewrite, apresentado na Seção 2.1; neles, os modelos principais são pagos, abertos de grande porte ou, no SPA, ajustados para a tarefa. O estudo de Amaral, Andrade e Campos (2025), que não propõe sistema, fica fora da tabela.

**Tabela 1. Comparativo entre os trabalhos relacionados**

| Trabalho | Modelo principal | Verificação de equivalência | Contexto do banco no *prompt* | Métrica de ganho | SGBD |
|---|---|---|---|---|---|
| LLM-R2 | GPT-3.5-turbo | por construção (o modelo só escolhe regras do Calcite) | uma demonstração e as regras candidatas; sem esquema, estatísticas ou plano | tempo medido | não declarado (regras no Calcite) |
| R-Bot | GPT-4o | por construção (regras do Calcite) | evidências recuperadas de documentação, do código das regras e de fóruns; custo usado na reflexão | tempo medido | PostgreSQL 14 |
| QUITE | DeepSeek-R1 e Claude-3.7-Sonnet | prova formal (SQLSolver) e, quando ela não decide, verificação pelo próprio modelo; comparação de resultados na avaliação | estatísticas, plano e custo do `EXPLAIN`, conhecimento recuperado | tempo medido | PostgreSQL 14 |
| LITHE | GPT-4o | amostras do banco, prova formal (QED, SQLSolver) e banco inteiro quando a prova não decide; decisão final do DBA | esquema, restrições, seletividades estimadas e exemplos de reescrita | custo estimado (principal) e tempo | PostgreSQL 16 e dois SGBDs comerciais |
| GenRewrite | o3-mini | prova formal (SQLSolver) e, quando ela não decide, teste por execução sobre entradas geradas; inspeção manual | regras em linguagem natural; plano com métricas de execução em chamada separada | tempo medido | PostgreSQL 14 |
| ReSequel | Gemini-2.5-pro e OSS-120B | execução sobre amostras do banco e comparação de resultados | esquema, índices, restrições e estatísticas | tempo medido | PostgreSQL 17, MySQL 8 e DuckDB |
| SPA | Qwen3-8B e Qwen3-32B com ajuste fino e aprendizado por reforço | execução e comparação de resultados | plano do `EXPLAIN` condensado em cardinalidades e seletividades | tempo medido | PostgreSQL 16 |
| **Presente trabalho** | Llama3.2 3B e llama3 8B locais, sem ajuste fino; GPT-4o | execução sobre a base inteira e comparação linha a linha | por fase: nada, esquema e número de linhas, catálogo de transformações, plano do `EXPLAIN` | custo estimado (principal) e tempo | PostgreSQL 14 |

## 4. Materiais e métodos

Este trabalho é uma pesquisa aplicada, de abordagem quantitativa e natureza experimental. O modelo apenas propõe a reescrita. Um validador a submete ao PostgreSQL e aplica regras fixas às respostas do banco: se ela executa, se devolve as mesmas linhas da original e se o custo estimado é menor. O modelo recebe texto, composto pela consulta e pelo contexto extraído via *script*, e devolve texto; não recebe credenciais nem tem conexão com o SGBD. Apenas o validador, descrito na Seção 4.6, executa SQL, original ou reescrito, em ambiente isolado. A reescrita classificada como otimizada é uma proposta ao desenvolvedor e a decisão de aplicação permanece com ele.

### 4.1. Bases de dados

O estudo usa duas bases com papéis distintos, sendo a denominada **base controlada** construída pelo autor com defeitos técnicos já conhecidos e a base **TPC-H**, que é referência da literatura (*baseline*) e possui consultas bem escritas/formuladas.

**Base controlada.** Um esquema próprio de loja, carregado com dados gerados por semente (*seed*) fixa para garantir reprodutibilidade: `cliente` (5.000 linhas), `produto` (500), `pedido` (50.000) e `item_pedido` (150.000), com índices nas colunas de junção e data, além da admissão de nulos pela coluna `pedido.cupom_id`. Sobre esse esquema, o estudo reúne 18 consultas deliberadamente ineficientes, sendo três por categoria de defeito, com categorias adaptadas da classificação de consultas mal escritas do ReSequel (Fathollahzadeh; Mansour; Boehm, 2026) e com a categoria de correção redefinida para armadilhas de equivalência (*equivalence pitfalls*) (ver Tabela 2). O código SQL de cada consulta, com o defeito e a reescrita esperada em comentário, está no repositório do protótipo[1].

Quinze consultas foram construídas para ter ganho e são denominadas, neste trabalho, como "otimizáveis". As três consultas de correção exercitam a verificação de resultado: `NOT IN` sobre coluna com nulos, `DISTINCT` necessário após junção e `COUNT(coluna)`, que ignora nulos; nelas não há ganho previsto, mas uma reescrita que preserve o resultado passa pelas mesmas regras das demais. A base própria também reduz o risco de reescritas memorizadas de bases públicas.

**TPC-H.** A base de referência da literatura (Transaction Processing Performance Council, 2022), em fator de escala 1, carregada em banco separado no mesmo contêiner, com chaves, índices e estatísticas atualizadas. O TPC-H entra para situar o estudo em relação à literatura, que avalia reescrita sobre bases de referência públicas, e para contrastar consultas já bem escritas com as da base controlada. O subconjunto tem quatro consultas oficiais, escolhidas por exercitarem as mesmas construções das categorias da base controlada: a Q03 junta três tabelas por junção implícita, com agregação e ordenação; a Q04 testa a existência de itens com uma subconsulta correlacionada em `EXISTS`; a Q06 filtra uma única tabela por faixas de data, desconto e quantidade; e a Q18 combina uma subconsulta em `IN` com `GROUP BY` e `HAVING` e uma junção implícita de três tabelas. Por ser cenário de contraste, o TPC-H ficou restrito a essas quatro consultas, que somam 144 execuções com cerca de um minuto de inferência por reescrita no llama3 (8B) (Seção 5.1), e as comparações estatísticas usam apenas a base controlada.

[1] Esquema, dados, consultas e código do protótipo:
`https://github.com/EduTremea/TCC/tree/main/Execução/prototipo`

**Tabela 2. Categorias de consultas na base controlada.**

| Categoria | Defeito | Reescrita esperada |
|---|---|---|
| sargabilidade[1] | função ou aritmética sobre a coluna filtrada impede o uso do índice | predicado de faixa ou comparação direta |
| subconsulta | subconsulta correlacionada executada por linha | junção com agrupamento ou semi-junção |
| junção | junção implícita ou junção sem uso | junção explícita ou remoção da junção |
| redundância | `DISTINCT` desnecessário, `UNION` de filtros exclusivos, `COUNT(DISTINCT)` sobre chave | projeção direta, `IN`, `COUNT(*)` |
| agregação | agregação escalar correlacionada | agregação única com `GROUP BY` |
| correção | consulta que parece ineficiente, mas cuja reescrita óbvia altera o resultado | nenhuma prevista; a reescrita óbvia deve ser rejeitada |

[1] Sargabilidade (*sargability*) indica se uma consulta SQL foi escrita de forma a permitir que o otimizador do banco de dados use um índice disponível para acelerar a busca

*Nota*: A categoria **correção** testa se o validador rejeita a reescrita óbvia que altera o resultado.

### 4.2. Ambiente experimental

Os experimentos rodaram em uma máquina com 16 GB de memória RAM e processador Intel Core i7-1355U, de 10 núcleos e 12 *threads*, sem GPU dedicada. O SGBD foi PostgreSQL 14.24 em *container* Docker, mesma versão principal dos trabalhos de referência (Sun *et al.*, 2025; Song *et al.*, 2025), com `shared_buffers` de 512 MB, `work_mem` de 64 MB e `effective_cache_size` de 2 GB, valores fixados antes da coleta. O protótipo fez uso da linguagem Python 3.12, sendo que os modelos de LLMs locais rodaram pelo Ollama, em CPU e sem rede. O protótipo não fixa o parâmetro `num_ctx` e usa a janela de contexto padrão do servidor, que variou entre versões do Ollama, de 2.048 *tokens* nas mais antigas a 4.096 ou mais nas recentes; o maior *prompt* do estudo, de 1.439 *tokens*, coube em qualquer uma delas. Os pesos do llama3.2 (3B) e do llama3 (8B) têm resumo SHA-256 iniciado por `dde5aa3fc5ff` e `6a0746a1ec1a`; a versão do Ollama usada de 5 a 12 de outubro de 2026 não foi registrada, e a instalada em 27 de outubro é a 0.40.11. O modelo pago é acessado pela API do fornecedor, sem ferramentas e sem busca na web.

### 4.3. Modelos de linguagem comparados

O `llama3.2`, com 3 bilhões de parâmetros, e o `llama3`, com 8 bilhões, são modelos abertos da família Llama, executados pelo Ollama em versões quantizadas de 4 bits (Q4_K_M e Q4_0, respectivamente); cabem na memória de uma máquina sem GPU e não têm custo por chamada, o cenário de quem não pode enviar dados a um serviço externo nem treinar localmente um modelo em GPU. O modelo pago é o `gpt-4o` (GPT-4o), identificador

exato enviado à API, escolhido por ser o modelo principal do LITHE (Dharwada *et al.*, 2026) e do R-Bot (Sun *et al.*, 2025) e pelo custo, de cerca de US$ 3,50 em todo o estudo. As chamadas ao GPT-4o que compõem os resultados foram feitas entre 7 e 12 de outubro de 2026. Todos os modelos rodaram com o parâmetro de temperatura (*AI temperature*) zero, visando respostas mais focadas e determinísticas. No restante do texto, cada modelo é designado por llama3.2 (3B), llama3 (8B) ou GPT-4o na primeira menção de cada seção e, nas demais, por 3B, 8B ou GPT-4o.

### 4.4. Cenários e fases

Os dois cenários propostos, usando a base de dados TPC-H e a base controlada, passam pela mesma sequência de quatro fases, chamadas C0 a C3. As fases são etapas, e não consultas: cada consulta passa pelas quatro, e cada fase acrescenta contexto ao *prompt* da anterior. Elas diferem apenas pelo que o modelo recebe com a consulta: na C0, há apenas uma instrução de reescrita, sendo o que a literatura chama de *zero-shot* (Brown *et al.*, 2020); na C1, é adicionado o esquema das tabelas, com colunas e tipos, e o número aproximado de linhas; na C2, além disso, um catálogo de oito transformações clássicas, em linguagem genérica (Apêndice A); e na C3, ainda o plano estimado da consulta original, obtido com `EXPLAIN`, como no QUITE, no GenRewrite e no SPA (Song *et al.*, 2025; Liu; Mozafari, 2025; Huang; Miao, 2026).

Todo o contexto é extraído do catálogo do PostgreSQL e entregue como texto, sem nenhuma linha de dado, e um teste automatizado garante que nenhum *prompt* cita os sistemas de referência, de modo que o modelo recebe apenas o contexto do banco de dados. Como variante de controle, a fase C1 foi executada também com esquema completo, com colunas `NOT NULL`, chaves e índices, uma vez por modelo.

### 4.5. Protocolo de execução

Uma execução corresponde a uma consulta, uma fase e um modelo e é uma chamada independente, sem histórico de conversa. O protótipo registra o *prompt* completo, a reescrita e o resultado de cada verificação. Cada combinação foi executada três vezes, porque o GPT-4o, acessado por API, não é determinístico mesmo com temperatura zero e porque a repetição mede a repetibilidade dos modelos locais: são $4 \times 4 \times 3 \times 3 = 144$ execuções no TPC-H e $18 \times 4 \times 3 \times 3 = 648$ na base controlada. As reescritas foram gravadas primeiro e medidas depois, sem nenhum modelo em execução.

### 4.6. Análise dos resultados

**Classificação.** Cada execução recebe exatamente uma de cinco categorias:

1. **não executa**: a reescrita não produz resultado, porque o PostgreSQL a recusa no `EXPLAIN`, por erro de sintaxe, referência a tabela ou coluna inexistente ou ambígua ou outra regra verificada antes da execução, como coluna fora do `GROUP BY`, ou porque a execução falha, por estouro de tempo (cinco minutos) ou de disco. É a mesma verificação que o LLM-R2 e o QUITE empregam para garantir que a reescrita é executável (Li; Yuan *et al.*, 2024; Song *et al.*, 2025).
2. **inválida por resultado**: a reescrita executa, mas devolve linhas diferentes das da consulta original. Original e reescrita são executadas por inteiro e lidas como tabelas, que precisam ter o mesmo número de linhas e de colunas e os

mesmos nomes de coluna; as colunas são postas em ordem alfabética e as linhas, ordenadas por todas as colunas, de modo que a ordem, inclusive a imposta por `ORDER BY`, não conta, mas as linhas repetidas contam, como em um multiconjunto. Os valores são comparados por igualdade exata, sem tolerância para ponto flutuante e com o mesmo tipo de dado em cada coluna, e dois nulos na mesma posição contam como iguais. Esta categoria é descartada sempre, independentemente de custo ou tempo, porque, sem o validador, esse erro passaria despercebido em uso real.

3. **sem mudança**: a reescrita é a própria consulta original, ignorando espaços, caixa e ponto e vírgula. Não é erro nem ganho e, por decisão de projeto, é contada em separado, para que uma reescrita idêntica nunca seja confundida com uma otimização.

4. **válida sem ganho**: mesmo resultado, mas custo estimado igual ou maior que o da original, categoria que o SPA também separa (Huang; Miao, 2026).

5. **otimizada**: mesmo resultado e custo estimado menor.

**Custo estimado como critério.** A medida de ganho é o custo total do plano estimado pelo planejador do PostgreSQL, obtido com `EXPLAIN` a partir das estatísticas das tabelas e de constantes de custo por página e por linha (The PostgreSQL Global Development Group, 2021a,b). É a medida usada no LITHE (Dharwada *et al.*, 2026) e no sistema de reescrita aprendida de Zhou, Li *et al.* (2021), trazendo os seguintes benefícios: independe do hardware, expressa a decisão do otimizador e, por ser determinística, não tem ruído de medição, de modo que não se aplica limiar ao custo, ao contrário do tempo. Em compensação, herda os erros de estimativa discutidos na Seção 2.1, e por isso o tempo é medido ao lado. O ganho de custo é a razão entre o custo da original e o da reescrita; o efeito de exigir um ganho mínimo é relatado na Seção 5.2.

**Tempo como complemento.** Toda execução válida tem também a razão entre o tempo da original e o da reescrita, medida sem nenhum modelo de linguagem carregado. O aquecimento é uma execução de cada consulta, descartada, para reduzir o efeito da partida a frio, como no QUITE e no SPA (Song *et al.*, 2025; Huang; Miao, 2026). Seguem-se cinco pares intercalados em ordem alternada, e a razão reportada é a mediana das cinco razões, decisão deste estudo para reduzir o peso de valores extremos, a mesma preocupação do R-Bot, que descarta o maior e o menor de cinco tempos (Sun *et al.*, 2025). O limiar para considerar a reescrita mais rápida é 1,15, acima da razão máxima de 1,148 observada ao submeter cada uma das 18 consultas da base controlada como se fosse a sua própria reescrita, em três rodadas, num total de 54 medições; o mesmo limiar foi aplicado ao TPC-H.

**Comparação entre modelos.** Um teste de sinais pareado por consulta compara, em cada uma das 18 consultas, o número de execuções otimizadas de dois modelos, exclui os empates e calcula o $p$-valor binomial bicaudal. As três comparações no total das fases são as planejadas e recebem a correção de Holm (para controlar falsos positivos); as comparações por fase são exploratórias. Nas tabelas, os números por fase são médias das três repetições, e os de síntese, somas das quatro fases e das três repetições.

### 4.7. Ameaças à validade

**Validade interna.** A verificação de resultado por execução não constitui prova de equivalência. O custo estimado é uma previsão do planejador e não captura diferenças em um mesmo plano, e o tempo, medido em máquina local com cache quente (*hot cache*), é só indicação. O GPT-4o não é determinístico mesmo com temperatura zero (mais detalhes na Seção 5.3). O protótipo registrou o identificador enviado à API, `gpt-4o`, e não a versão devolvida na resposta; como esse identificador é um apelido para uma versão datada, e só as versões datadas fixam o modelo (OpenAI, 2026b), os resultados valem para a versão servida entre 7 e 12 de outubro de 2026. As versões datadas do GPT-4o têm remoção da API prevista para 14 de janeiro de 2027 (OpenAI, 2026a), e depois dessa data essa parte do experimento não poderá ser repetida. Como as fases são cumulativas, o efeito de cada bloco de contexto não se separa do aumento do *prompt*. Os modelos locais rodaram quantizados em 4 bits.

**Validade externa.** São 18 consultas sintéticas e quatro do TPC-H, o que limita a generalização e o poder do teste; o TPC-H provavelmente está no material de treinamento dos modelos, risco que a base própria reduz. O formato de instrução foi um só, a variante de esquema completo tem uma repetição e os modelos avaliados não são os mais recentes.

**Validade de construto.** O próprio autor escreveu as consultas, a reescrita esperada e o catálogo da fase C2, o que pode alinhá-los; a reescrita esperada não foi fornecida aos modelos nem usada na classificação.

### 4.8. Uso de ferramentas de IA

O desenvolvimento do código do protótipo e a revisão do texto tiveram apoio de um assistente de IA, usado para escrever e depurar código, organizar os dados gravados e revisar a redação. O desenho experimental, a escolha das consultas e das categorias, a definição das fases e da escala de classificação, o protocolo de medição e a análise dos resultados são do autor, sob orientação. Os modelos de linguagem avaliados no experimento são tratados como objetos de estudo e não participaram da escrita do artigo.

## 5. Resultados

Os dois cenários são apresentados na seguinte ordem: primeiro a base de referência, depois a base controlada. As tabelas agregadas, os arquivos CSV de origem e a planilha com todas as execuções estão no repositório[2].

### 5.1. Cenário 1: TPC-H

As 144 execuções do TPC-H estão classificadas por fase na Figura 1. Nenhum modelo otimizou nenhuma consulta em nenhuma fase: toda reescrita válida teve o mesmo custo estimado e o mesmo plano de execução da original. Das 144 execuções, 91 foram válidas sem ganho, 26 invalidaram o resultado e 27 não executaram. A única execução com razão de tempo igual ou superior a 1,15 foi uma repetição de Q06 pelo llama3 (8B) na fase C0, com custo idêntico ao da original. Todas as reescritas do GPT-4o executaram, em todas as fases; no 8B, nove das doze execuções da fase C3 não executaram: nas três repetições da Q03, da Q04 e da Q18, a reescrita reestruturou a consulta e referenciou coluna inexistente, ou ambígua, ou deixou coluna fora do `GROUP BY`.

[2] `https://github.com/EduTremea/TCC/tree/main/Execução/resultados`

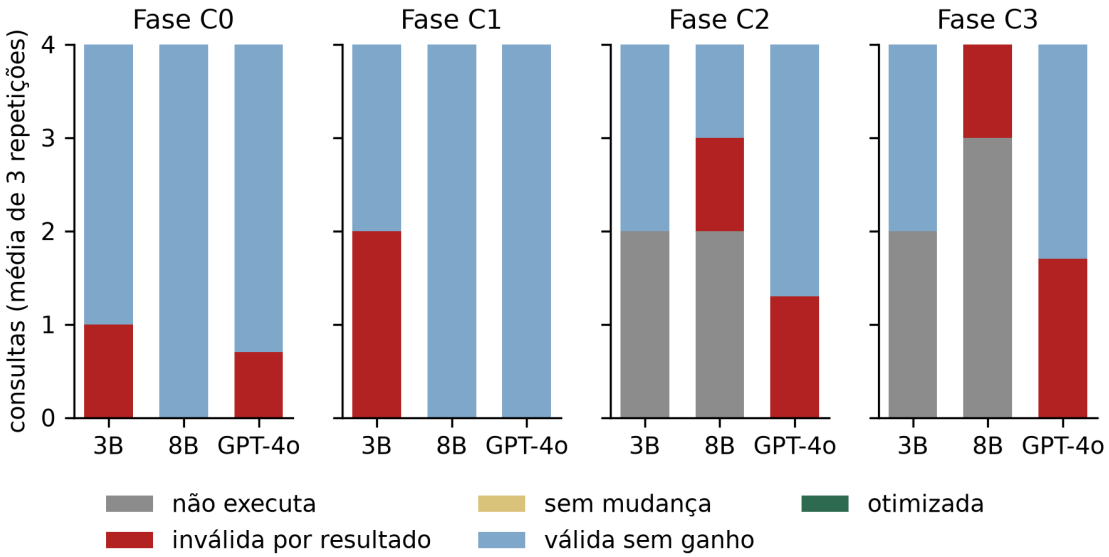



**Figura 1. Classificação das execuções no TPC-H por fase e modelo. Valor máximo no eixo Y é 4.**

### 5.2. Cenário 2: base controlada

As 18 consultas passaram pela mesma sequência, totalizando 648 execuções (Figura 2). Na fase C0, o llama3.2 (3B) devolveu a própria consulta original em seis das 18 consultas; o esquema da fase C1 elevou as otimizadas do 8B e do GPT-4o, e as fases C2 e C3 aumentaram as inválidas por resultado dos dois modelos locais. Em nenhuma fase o GPT-4o teve reescrita que não executasse ou que devolvesse a consulta original sem mudança. O Apêndice B mostra uma execução completa, da consulta original à classificação.

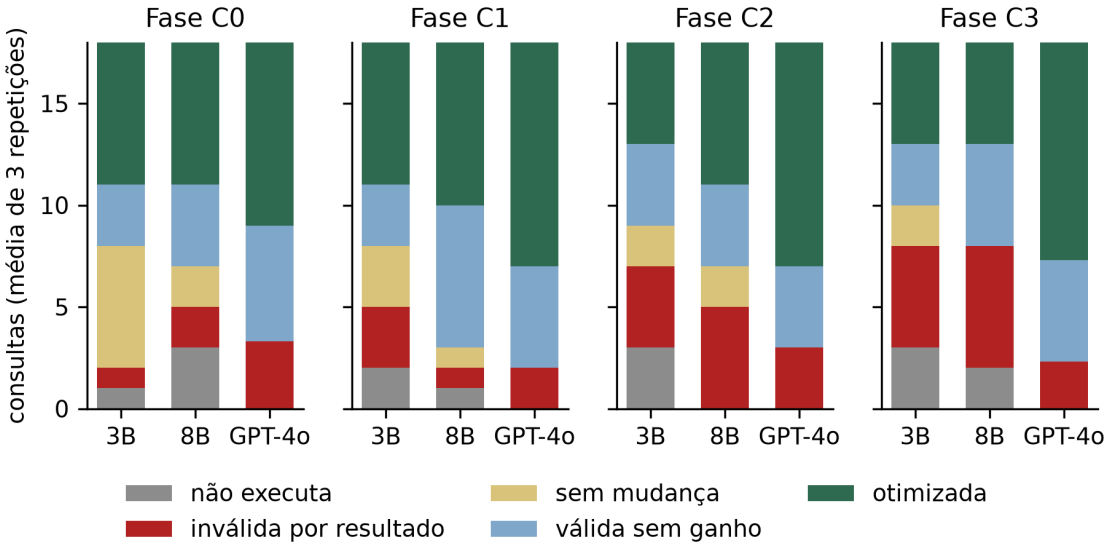



**Figura 2. Classificação das execuções na base controlada por fase e modelo. Valor máximo no eixo Y é 18.**

**Reescritas inválidas.** Na `correcao_01`, armadilha do `NOT IN` sobre coluna com

nulo, o 8B trocou, na fase C1 e nas três repetições, o `NOT IN` por `NOT EXISTS`. Como há pedidos pagos sem cupom, a lista do `NOT IN` contém um nulo e a original devolve 0 linhas, enquanto o `NOT EXISTS`, que não é afetado pelo nulo, devolve 4.901; o validador classificou a reescrita como inválida por resultado, e os três modelos tiveram execuções inválidas nessa consulta. A Tabela 3 classifica as 113 execuções inválidas por resultado pelo tipo de diferença. Elas correspondem a 38 reescritas distintas; das 113 execuções, 52 vêm das três consultas de correção, 30 de agregação, 21 de subconsulta, 6 de redundância, 3 de junção e 1 de sargabilidade.

**Tabela 3. Execuções inválidas por resultado na base controlada, por tipo de diferença em relação à consulta original.**

| Tipo de diferença | llama3.2 (3B) | llama3 (8B) | GPT-4o | Total |
|---|---|---|---|---|
| mais linhas que a original | 18 | 24 | 16 | 58 |
| menos linhas que a original | 6 | 9 | 12 | 27 |
| mesmo número de linhas, valores diferentes | 15 | 9 | 4 | 28 |
| **Total** | **39** | **42** | **32** | **113** |

*Nota*: Cada número é uma soma das quatro fases e das três repetições.

**Otimizadas e cobertura.** Sob o critério de custo (Tabela 4), o GPT-4o otimizou 58% das execuções, o 8B, 38%, e o 3B, 33%, e exigir também a razão de tempo de 1,15 reduz todas as contagens sem mudar a ordem. O GPT-4o otimizou dez das 15 consultas otimizáveis e cada modelo local, oito, todas entre as dez do GPT-4o; só ele otimizou uma junção, a `juncao_02`, e, na fase C1, as três consultas de sargabilidade nas três repetições. Nas consultas otimizáveis que cobre, excluída a `correcao_03`, que não é otimizável, o GPT-4o otimizou 116 de 120 execuções; o 8B, 78 de 96; o 3B, 72 de 96. Somadas as execuções otimizadas da `correcao_03`, 9 no GPT-4o e 3 no 8B, chega-se aos totais de 125, 81 e 72 da Tabela 7. Cinco consultas não foram otimizadas por nenhum modelo: `agregacao_01`, `juncao_01`, `juncao_03`, `redundancia_03` e `subconsulta_03`.

**Tabela 4. Consultas otimizadas por modelo e fase, sob os critérios de custo e custo combinado com tempo.**

| Modelo | Critério | C0 | C1 | C2 | C3 | Total (72) | Taxa |
|---|---|---|---|---|---|---|---|
| llama3.2 (3B) | otimizada (custo) | 7,0 | 7,0 | 5,0 | 5,0 | 24,0 | 33% |
| llama3 (8B) | otimizada (custo) | 7,0 | 8,0 | 7,0 | 5,0 | 27,0 | 38% |
| GPT-4o | otimizada (custo) | 9,0 | 11,0 | 11,0 | 10,7 | 41,7 | 58% |
| llama3.2 (3B) | otimizada e tempo $\geq 1{,}15$ | 5,0 | 4,3 | 3,0 | 4,0 | 16,3 | 23% |
| llama3 (8B) | otimizada e tempo $\geq 1{,}15$ | 5,0 | 5,7 | 6,0 | 4,0 | 20,7 | 29% |
| GPT-4o | otimizada e tempo $\geq 1{,}15$ | 6,0 | 7,7 | 7,7 | 7,7 | 29,1 | 40% |

*Nota*: Cada número é quantas das 18 consultas foram otimizadas naquela fase, na média das três repetições. O total soma as quatro fases (72 consultas), e a taxa é o total dividido por 72.

**Comparação entre modelos.** Para cada uma das 18 consultas, compara-se o número de execuções otimizadas de cada modelo (soma das quatro fases e das três repetições); "A $>$ B" conta em quantas consultas o primeiro modelo superou o segundo, os empates são excluídos e o $p$-valor é binomial bicaudal. No total das quatro fases (Tabela 5),

o GPT-4o superou cada modelo local em todas as consultas em que houve diferença; com a correção de Holm nas três comparações planejadas, os $p$-valores ajustados são 0,023 contra o 3B, 0,031 contra o 8B e 0,45 entre os locais. Em nenhuma fase e em nenhuma consulta um modelo local superou o GPT-4o, mas, com 12 a 16 empates em 18 consultas, os testes por fase têm pouco poder; na fase C1, nenhum é significativo. A significância depende de não haver ganho mínimo: exigindo razão de custo de pelo menos 1,05, a vantagem sobre o 8B deixa de ser significativa após a correção ($p$ ajustado 0,062, contra 0,047 para o 3B), e com 1,10 nenhuma comparação se mantém, embora a ordem entre os modelos não mude.

**Tabela 5. Teste de sinais pareado por consulta, sob o critério de custo.**

| Escopo | Comparação | A > B | A < B | Empates | $p$-valor |
|---|---|---|---|---|---|
| C1 | GPT-4o x llama3 (8B) | 3 | 0 | 15 | 0,25 |
| C1 | GPT-4o x llama3.2 (3B) | 4 | 0 | 14 | 0,13 |
| C1 | llama3 (8B) x llama3.2 (3B) | 2 | 1 | 15 | 1,00 |
| Total | GPT-4o x llama3 (8B) | 7 | 0 | 11 | 0,016 |
| Total | GPT-4o x llama3.2 (3B) | 8 | 0 | 10 | 0,008 |
| Total | llama3 (8B) x llama3.2 (3B) | 5 | 2 | 11 | 0,45 |

**Ganho de custo.** A Tabela 6 lista as onze consultas com pelo menos uma execução otimizada. Dez delas foram otimizadas por mais de um modelo e, em oito, a razão entre o maior e o menor ganho é de no máximo 1,06. As exceções são a `redundancia_01`, em que o 3B ficou muito abaixo dos demais (9,2 contra 784 e 934), e a `subconsulta_01`, em que o 8B ficou abaixo dos outros dois (2,57 contra 16,7 e 17,6). Quatro consultas concentram os maiores ganhos, de 88,9 a 934 (exceto o 3B na `redundancia_01`); nas demais, o ganho vai de 1,02 a 17,6. A `correcao_03`, que é da categoria de correção, aparece porque o GPT-4o e o 8B preservaram o resultado ao acrescentar o filtro `cupom_id IS NOT NULL`, enquanto a reescrita óbvia, `COUNT(*)` sem o filtro, foi invalidada. A mediana do ganho, calculada sobre todas as execuções otimizadas de cada modelo, foi de 14,8 no 3B, 88,9 no 8B e 2,6 no GPT-4o.

**Custo e tempo.** Sob o critério só de tempo, a ordem entre os modelos no total é a mesma do custo; nas fases C0 e C3, em que os locais empatam sob custo, o 8B fica à frente sob tempo. Algumas reescritas válidas sem ganho de custo passaram de 1,15 no tempo. O caso recorrente é a `redundancia_03`: a troca de `COUNT(DISTINCT id)` por `COUNT(*)` mantém o plano e o custo estimado, 958, e executa entre 30% e 50% mais rápido nas três repetições, no 8B e no GPT-4o.

**Síntese do cenário.** Em 648 execuções (Tabela 7), 113 reescritas invalidaram o resultado e 54 devolveram a própria consulta original. A coluna "sem efeito" reúne duas categorias, sem mudança e válida sem ganho, que somam, respectivamente, 39 e 39 execuções no 3B, 15 e 60 no 8B e 0 e 59 no GPT-4o. Das 27 execuções do 3B na categoria "não executa", 6 foram reescritas com produto cartesiano que ultrapassaram o limite de cinco minutos. O tempo de inferência, do envio do *prompt* ao recebimento da reescrita, teve mediana de 7,1 s no 3B, 16,6 s no 8B e 0,8 s no GPT-4o na base controlada, e de 24,7 s, 57,3 s e 1,5 s no TPC-H. Nos modelos locais, esse tempo cresce com o tamanho do *prompt*; no GPT-4o, inclui a ida e volta pela rede.

**Tabela 6. Ganho de custo por consulta e modelo: mediana da razão entre o custo da original e o da reescrita.**

| Consulta | llama3.2 (3B) | llama3 (8B) | GPT-4o | Maior / menor |
|---|---|---|---|---|
| agregacao_02 | 88,9 | 88,9 | 94,3 | 1,06 |
| agregacao_03 | 1,12 | – | 1,12 | 1,00 |
| correcao_03 | – | 1,02 | 1,02 | 1,00 |
| juncao_02 | – | – | 1,24 | – |
| redundancia_01 | 9,2 | 784 | 934 | 101 |
| redundancia_02 | 2,57 | 2,57 | 2,57 | 1,00 |
| sargabilidade_01 | – | 1,11 | 1,11 | 1,00 |
| sargabilidade_02 | 1,05 | 1,05 | 1,05 | 1,00 |
| sargabilidade_03 | 265 | 265 | 265 | 1,00 |
| subconsulta_01 | 17,6 | 2,57 | 16,7 | 6,84 |
| subconsulta_02 | 160 | 158 | 159 | 1,01 |

*Nota*: A mediana considera as execuções otimizadas daquela consulta, em todas as fases e repetições. "–" indica que o modelo não otimizou a consulta. A última coluna é a razão entre o maior e o menor ganho entre os modelos que a otimizaram.

**Tabela 7. Síntese por modelo na base controlada, com 216 execuções por modelo.**

| Modelo | Melhorou (otimizada) | Sem efeito | Invalidou o resultado | Não executa |
|---|---|---|---|---|
| llama3.2 (3B) | 72 (33%) | 78 (36%) | 39 (18%) | 27 (12%) |
| llama3 (8B) | 81 (38%) | 75 (35%) | 42 (19%) | 18 (8%) |
| GPT-4o | 125 (58%) | 59 (27%) | 32 (15%) | 0 |

*Nota*: Cada número é a soma das quatro fases e das três repetições. "Sem efeito" soma as categorias sem mudança e válida sem ganho.

### 5.3. Repetibilidade

Com temperatura zero, o llama3 (8B) devolveu o mesmo SQL nas três repetições em todas as 72 células da base controlada, e o llama3.2 (3B), em 70; nas outras duas, uma repetição diferiu sem mudar a categoria, e a causa não foi investigada. O GPT-4o devolveu o mesmo SQL em 60 células. Como o custo estimado é determinístico para um mesmo SQL, a variação sob o critério de custo vem só do GPT-4o: a categoria mudou em duas células, a `correcao_02` na fase C0 e a `sargabilidade_02` na fase C3, em que uma das três repetições foi inválida por resultado. Na leitura por tempo, o veredito de razão $\geq 1{,}15$ mudou entre repetições com o mesmo SQL em uma célula do 3B, três do 8B e uma do GPT-4o, efeito do ruído de medição que o custo estimado não tem.

## 6. Discussão

**Validação.** O achado mais estável do estudo é a proporção de reescritas que alteram o resultado: de 15% a 19% das execuções de cada modelo na base controlada, inclusive no GPT-4o, cujas reescritas sempre executaram. Uma verificação que só testasse se a

reescrita executa aceitaria todas elas. Nas três consultas de correção, construídas para que a reescrita óbvia alterasse o resultado, essa reescrita apareceu e foi sempre barrada, e as aceitas mantiveram o que a armadilha exige: o `NOT IN` na `correcao_01`, a eliminação de duplicatas, por `DISTINCT`, `GROUP BY` ou `EXISTS`, na `correcao_02`, e o filtro de nulos na `correcao_03`, a única que trouxe ganho de custo.

Entre o ganho e o erro, a leitura em cinco categorias mostra ainda uma parcela de reescritas sem efeito, de 27% das execuções no GPT-4o a 36% no llama3.2 (3B), segura em uso prático porque a consulta original é mantida.

O validador, porém, só assegura o mesmo resultado na instância avaliada. Resultados iguais sobre uma base não provam equivalência (Seção 2.2), e a `agregacao_02` mostra o efeito: a original soma os itens de cada produto numa subconsulta escalar, que devolve nulo para produto sem itens, e as 36 execuções foram aceitas como otimizadas, mas 32 trocam esse nulo por zero, com `COALESCE`, ou descartam o produto, com junção interna. Elas só passaram porque, nesta base, todo produto tem itens; apenas quatro, com junção externa e sem `COALESCE`, são equivalentes em geral. Na `agregacao_01`, em que há pedidos pagos sem itens, a mesma junção interna foi barrada, como se detalha adiante. As demais reescritas aceitas não passaram por prova formal nem por inspeção manual sistemática, e o risco é maior em bases pequenas e sintéticas como a deste estudo. Quanto às inválidas, quase metade vem das consultas de correção e o restante, sobretudo, de agregação e subconsulta, em que a troca de uma subconsulta correlacionada por junção pode multiplicar ou perder linhas; em 28 casos o número de linhas é igual ao da original (Tabela 3), o que uma verificação baseada só na contagem não detectaria.

**Modelo pago e modelos locais.** Sob o critério de custo, o llama3 (8B) obteve cerca de dois terços das otimizações do GPT-4o, 81 contra 125 execuções, e o llama3.2 (3B), 72. A vantagem do GPT-4o sobre cada local é significativa no total das fases, após a correção de Holm e sem ganho mínimo de custo, e os dois locais não diferem entre si (Tabela 5).

A cobertura explica a diferença: o GPT-4o otimiza todas as consultas que algum local otimiza, mais a `juncao_02`, e acerta com mais regularidade nas que cobre. Das cinco consultas que nenhum modelo otimizou, duas são junções implícitas cuja forma explícita produz o mesmo plano, uma usa um `IN` que o planejador já trata como semijunção e outra, o `COUNT(DISTINCT)` sobre chave, tem reescrita que não altera o plano. Na quinta, a `agregacao_01`, que soma os itens de cada pedido pago numa subconsulta escalar, os modelos a trocaram por junção interna, que descarta os pedidos pagos sem itens: as 12 execuções do GPT-4o e 6 do 8B foram inválidas por resultado, com 28.482 linhas contra 29.949, e as outras 18 não executaram. A reescrita correta exige junção externa, que nenhum modelo propôs.

O ganho de custo depende mais da consulta do que do modelo. Nas consultas otimizadas por mais de um modelo, as medianas praticamente coincidem na maioria dos casos (Tabela 6), porque os modelos convergem para a mesma reescrita ou para reescritas com o mesmo plano, e o teto do ganho é dado pela consulta, mais do que pela categoria do defeito: na sargabilidade, vai de 1,05 a 265. As medianas diferentes por modelo refletem a cobertura, e não a qualidade da reescrita: os três modelos otimizam as mesmas quatro consultas de ganho grande, salvo o 3B na `redundancia_01`, em que o ganho foi de 9,2,

e o GPT-4o otimiza também sete das demais, contra cinco do 8B e quatro do 3B.

Os perfis de erro também diferem: todas as reescritas do GPT-4o executaram e nenhuma repetiu a original sem mudança, enquanto o 3B devolveu a original sem alteração em 18% das execuções. Para o cenário em que dados e esquema não podem sair da organização, o 8B sob validação é a opção sem custo por chamada, mas o tempo de inferência limita o uso: sem GPU, o 8B leva cerca de 17 segundos por reescrita na base controlada e cerca de um minuto no TPC-H, o que é compatível com a revisão de uma consulta lenta por vez, mas não com reescrever uma carga inteira em lote.

**Contexto.** A segunda pergunta do estudo era se um modelo local, recebendo mais contexto, alcança o modelo pago. Isso não ocorreu em nenhuma fase: em todas, os locais otimizaram menos execuções que o GPT-4o, e em nenhuma consulta um local o superou. O esquema e os tamanhos elevaram as otimizadas do GPT-4o de 9,0 para 11,0 e as do 8B de 7,0 para 8,0, variações de uma ou duas consultas que o teste de sinais não distingue do acaso; o 3B ficou em 7,0. O catálogo e o plano de execução não acrescentaram otimizadas e aumentaram as inválidas por resultado dos locais, e o esquema completo, em uma única repetição na fase C1, reduziu as otimizadas dos locais, de 7,0 para 5,0 no 3B e de 8,0 para 7,0 no 8B, e manteve as 11,0 do GPT-4o. Uma hipótese compatível com esses dados é que, nos modelos pequenos, as transformações sugeridas passam a ser aplicadas mesmo quando a condição de equivalência não vale. Na `correcao_02`, por exemplo, o 8B manteve o `DISTINCT` nas fases C0 e C1 e o removeu na fase C2, como o catálogo sugere para colunas projetadas já únicas, embora `c.uf` não seja única; o resultado passou de 5 para 29.949 linhas nas três repetições. Como as fases são cumulativas e o formato de instrução foi um só, o estudo não separa o efeito de cada bloco do aumento do *prompt*.

**Cenários e trabalhos relacionados.** No TPC-H, em consultas já bem escritas, não restou transformação a aplicar; o ganho apareceu na base de consultas com defeitos conhecidos, o cenário em que um desenvolvedor submete uma consulta lenta para revisão. O método é, portanto, uma ferramenta de revisão de consultas, não um substituto do otimizador.

O que este estudo acrescenta é a comparação, sob a mesma validação, de modelos locais pequenos, sem ajuste fino e executados em CPU, com o GPT-4o, modelo principal do R-Bot e do LITHE. A resposta, cerca de dois terços das otimizações do GPT-4o, vale para os modelos e o formato de instrução avaliados. O resultado nulo no TPC-H também delimita a comparação: os trabalhos relacionados avaliam conjuntos maiores de consultas, vários deles com busca sobre alternativas ou com várias chamadas ao modelo, enquanto aqui são quatro consultas em fator de escala 1 e cada reescrita vem de uma única chamada.

## 7. Considerações Finais

Este trabalho avaliou a reescrita de consultas SQL por modelos de linguagem sob uma metodologia em que o modelo apenas sugere a nova consulta e um validador a classifica em cinco categorias, a partir das respostas do próprio SGBD, comparando dois modelos locais e um modelo pago.

As reescritas foram classificadas como otimizadas em 33% a 58% das execuções da base controlada, conforme o modelo, e em nenhuma do TPC-H; de 15% a 19% das execuções de cada modelo alteraram o resultado e foram barradas pelo validador. A com-

paração de resultados assegura o mesmo resultado na instância avaliada, e não a equivalência em geral, como mostra a `agregacao_02`. Nenhum modelo local alcançou o modelo pago em nenhuma fase, mas o llama3 (8B) entregou cerca de dois terços das otimizações do GPT-4o, 38% contra 58% das execuções, sem custo por chamada e sem enviar dado ou esquema a serviço externo. A diferença é significativa no total das fases após a correção de Holm, desde que não se exija ganho mínimo de custo, e entre os dois modelos locais não há diferença significativa.

O contexto fornecido não compensou a diferença entre os modelos: o esquema e os tamanhos elevaram pouco as otimizadas do GPT-4o e do 8B, e o catálogo e o plano de execução aumentaram as reescritas inválidas dos locais. O ganho variou mais entre consultas do que entre modelos e se concentra em consultas mal escritas, o que situa o método como ferramenta de revisão de consultas lentas, não como substituto do otimizador. O procedimento mostrou-se repetível: sob o critério de custo, a classificação só mudou entre repetições em duas células do GPT-4o, cuja API não é determinística.

As limitações estão na Seção 4.7, às quais se soma o tempo de inferência dos modelos locais em CPU. Como trabalhos futuros, indica-se usar GPU com capacidade compatível para treinamento de LLMs, ampliar o TPC-H, incluir o TPC-DS (Nambiar; Poess, 2006) e consultas reais de produção, complementar a comparação de resultados com casos de borda ou prova formal, avaliar modelos locais mais recentes e outros formatos de instrução e integrar o protótipo a um fluxo de desenvolvimento em que a reescrita otimizada chegue ao desenvolvedor como proposta.

## Referências


ABITEBOUL, S.; HULL, R.; VIANU, V. **Foundations of Databases**. Reading, MA: Addison-Wesley, 1995.

AMARAL, K. P. N. do; ANDRADE, L. P. de; CAMPOS, E. A. V. de. O uso da Inteligência Artificial na otimização de consultas em bancos de dados. **Revista Camalotes – RECAM**, v. 4, n. 1, p. 1–17, 2025. Disponível em: https://periodicos.insted.edu.br/recam/article/view/142. Acesso em: 9 jun. 2026.

BEGOLI, E. *et al.* Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. *In:* PROCEEDINGS of the 2018 International Conference on Management of Data (SIGMOD). [*S. l.*]: ACM, 2018. p. 221–230. DOI: 10.1145/3183713.3190662.

BROWN, T. B. *et al.* Language Models are Few-Shot Learners. *In:* ADVANCES in Neural Information Processing Systems 33 (NeurIPS). [*S. l.: s. n.*], 2020. p. 1877–1901. Disponível em: https://arxiv.org/abs/2005.14165. Acesso em: 16 jun. 2026.

CHAUDHURI, S. An Overview of Query Optimization in Relational Systems. *In:* PROCEEDINGS of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS). [*S. l.*]: ACM, 1998. p. 34–43. DOI: 10.1145/275487.275492.

CHU, S. *et al.* Cosette: An Automated Prover for SQL. *In:* 8TH Biennial Conference on Innovative Data Systems Research (CIDR). [*S. l.: s. n.*], 2017. Disponível em: https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf. Acesso em: 16 jun. 2026.

DHARWADA, S. *et al.* LITHE: A Query Rewrite Advisor using LLMs. *In:* INTERNATIONAL CONFERENCE ON EXTENDING DATABASE TECHNOLOGY. **Proceedings of the 29th International Conference on Extending Database Technology (EDBT 2026)**. Tampere, Finland: OpenProceedings.org, 2026. Disponível em: https://openproceedings.org/2026/conf/edbt/paper-93.pdf. Acesso em: 9 jun. 2026.

FATHOLLAHZADEH, S.; MANSOUR, E.; BOEHM, M. ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling. **Proceedings of the VLDB Endowment**, v. 19, n. 10, p. 2880–2893, 2026. Disponível em: https://arxiv.org/abs/2606.20853. Acesso em: 24 out. 2026.

HE, Y. *et al.* VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints. **Proceedings of the ACM on Programming Languages**, v. 8, OOPSLA1, p. 1–29, 2024. Disponível em: https://arxiv.org/abs/2403.03193. Acesso em: 16 jun. 2026.

HUANG, L.; YU, W. *et al.* A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. **ACM Transactions on Information Systems**, 2024. Disponível em: https://arxiv.org/abs/2311.05232. Acesso em: 16 jun. 2026.

HUANG, X.; MIAO, Z. **SPA: A SQL-Plan-Aware Reinforcement Learning Framework for Query Rewriting with LLMs**. [*S. l.: s. n.*], 2026. arXiv:2606.08620 [cs.DB]. Disponível em: https://arxiv.org/abs/2606.08620. Acesso em: 24 out. 2026.

LEIS, V. *et al.* How Good Are Query Optimizers, Really? **Proceedings of the VLDB Endowment**, v. 9, n. 3, p. 204–215, 2015.

LEWIS, P. *et al.* Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *In:* ADVANCES in Neural Information Processing Systems 33 (NeurIPS). [*S. l.: s. n.*], 2020. p. 9459–9474. Disponível em: https://arxiv.org/abs/2005.11401. Acesso em: 16 jun. 2026.

LI, G.; ZHOU, X.; ZHAO, X. LLM for Data Management. **Proceedings of the VLDB Endowment**, v. 17, n. 12, p. 4213–4216, 2024.

LI, J.; HUI, B. *et al.* Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. *In:* ADVANCES in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track. [*S. l.: s. n.*], 2023. Disponível em: https://arxiv.org/abs/2305.03111. Acesso em: 16 jun. 2026.

LI, Z.; YUAN, H. *et al.* LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. **Proceedings of the VLDB Endowment**, v. 18, n. 1, p. 53–65, 2024. Disponível em: https://www.vldb.org/pvldb/vol18/p53-yuan.pdf. Acesso em: 9 jun. 2026.

LIU, J.; MOZAFARI, B. **GenRewrite: Query Rewriting via Large Language Models**. [*S. l.: s. n.*], 2025. arXiv preprint, abs/2403.09060v3. DOI: 10.48550/arXiv.2403.09060. Disponível em: https://arxiv.org/abs/2403.09060. Acesso em: 16 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; TATBUL, N. *et al.* Bao: Making Learned Query Optimization Practical. *In:* ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA. **Proceedings of the 2021 International Conference on Management of Data**. Virtual Event, China: ACM, 2021. p. 1275–1288. DOI: 10.1145/3448016.3452838. Disponível em: https://people.csail.mit.edu/hongzi/content/publications/BAO-Sigmod21.pdf. Acesso em: 9 jun. 2026.

MARCUS, R.; NEGI, P.; MAO, H.; ZHANG, C. *et al.* Neo: A Learned Query Optimizer. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1705–1718, 2019. Disponível em: https://www.vldb.org/pvldb/vol12/p1705-marcus.pdf. Acesso em: 9 jun. 2026.

NAMBIAR, R. O.; POESS, M. The Making of TPC-DS. *In:* PROCEEDINGS of the 32nd International Conference on Very Large Data Bases (VLDB). [*S. l.*]: VLDB Endowment, 2006. p. 1049–1058.

OPENAI. **Deprecations**. [*S. l.*], 2026a. Disponível em: https://developers.openai.com/api/docs/deprecations. Acesso em: 28 out. 2026.

OPENAI. **GPT-4o Model**. [*S. l.*], 2026b. Disponível em: https://developers.openai.com/api/docs/models/gpt-4o. Acesso em: 28 out. 2026.

PIRAHESH, H.; HELLERSTEIN, J. M.; HASAN, W. Extensible/Rule Based Query Rewrite Optimization in Starburst. *In:* PROCEEDINGS of the 1992 ACM SIGMOD International Conference on Management of Data. [*S. l.*]: ACM, 1992. p. 39–48. DOI: 10.1145/130283.130294.

SELINGER, P. G. *et al.* Access Path Selection in a Relational Database Management System. *In:* PROCEEDINGS of the 1979 ACM SIGMOD International Conference on Management of Data. [*S. l.*]: ACM, 1979. p. 23–34. DOI: 10.1145/582095.582099.

SONG, Y. *et al.* **QUITE: A Query Rewrite System Beyond Rules with LLM Agents**. [*S. l.: s. n.*], 2025. arXiv preprint, abs/2506.07675. DOI: 10.48550/arXiv.2506.07675. Disponível em: https://arxiv.org/abs/2506.07675. Acesso em: 9 jun. 2026.

SUN, Z. *et al.* R-Bot: An LLM-based Query Rewrite System. **Proceedings of the VLDB Endowment**, v. 18, n. 12, p. 5031–5044, 2025. Disponível em: https://www.vldb.org/pvldb/vol18/p5031-li.pdf. Acesso em: 9 jun. 2026.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 14 Documentation: Chapter 14, Performance Tips, Section 14.1, Using EXPLAIN**. [*S. l.*], 2021a. Disponível em: https://www.postgresql.org/docs/14/using-explain.html. Acesso em: 24 out. 2026.

THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 14 Documentation: Chapter 20, Server Configuration, Section 20.7.2, Planner Cost Constants**. [*S. l.*], 2021b. Disponível em: https://www.postgresql.org/docs/14/runtime-config-query.html. Acesso em: 24 out. 2026.

TRANSACTION PROCESSING PERFORMANCE COUNCIL. **TPC Benchmark H (TPC-H) Standard Specification**. [*S. l.: s. n.*], 2022. Revision 3.0.1. Disponível em: https://www.tpc.org/tpch/. Acesso em: 16 jun. 2026.

WANG, X. *et al.* Self-Consistency Improves Chain of Thought Reasoning in Language Models. *In:* THE Eleventh International Conference on Learning Representations (ICLR). [*S. l.: s. n.*], 2023. Disponível em: https://arxiv.org/abs/2203.11171. Acesso em: 16 jun. 2026.

WEI, J. *et al.* Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *In:* ADVANCES in Neural Information Processing Systems 35 (NeurIPS). [*S. l.: s. n.*], 2022. p. 24824–24837. Disponível em: https://arxiv.org/abs/2201.11903. Acesso em: 16 jun. 2026.

YANG, Z. *et al.* NeuroCard: One Cardinality Estimator for All Tables. **Proceedings of the VLDB Endowment**, v. 14, n. 1, p. 61–73, 2021. Disponível em: https://www.vldb.org/pvldb/vol14/p61-yang.pdf. Acesso em: 9 jun. 2026.

YU, T. *et al.* Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *In:* PROCEEDINGS of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP). [*S. l.: s. n.*], 2018. p. 3911–3921. Disponível em: https://arxiv.org/abs/1809.08887. Acesso em: 16 jun. 2026.

ZHOU, Q.; ARULRAJ, J.; NAVATHE, S. B.; HARRIS, W.; WU, J. SPES: A Symbolic Approach to Proving Query Equivalence Under Bag Semantics. *In:* 2022 IEEE 38th International Conference on Data Engineering (ICDE). [*S. l.*]: IEEE, 2022. p. 2735–2748. DOI: 10.1109/ICDE53745.2022.00250.

ZHOU, Q.; ARULRAJ, J.; NAVATHE, S. B.; HARRIS, W.; XU, D. Automated Verification of Query Equivalence Using Satisfiability Modulo Theories. **Proceedings of the VLDB Endowment**, v. 12, n. 11, p. 1276–1288, 2019.

ZHOU, X.; LI, G. *et al.* A Learned Query Rewrite System using Monte Carlo Tree Search. **Proceedings of the VLDB Endowment**, v. 15, n. 1, p. 46–58, 2021. Disponível em: https://www.vldb.org/pvldb/vol15/p46-li.pdf. Acesso em: 24 out. 2026.

ZHU, R. *et al.* FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. **Proceedings of the VLDB Endowment**, v. 14, n. 9, p. 1489–1502, 2021. Disponível em: https://doi.org/10.14778/3461535.3461539. Acesso em: 9 jun. 2026.

## APÊNDICE A – Prompts usados nas fases de contexto

O texto abaixo reproduz o que foi enviado aos modelos, sem acentos, como no protótipo. A instrução é a mesma em todas as fases e para todos os modelos:

```
Voce reescreve consultas SQL para PostgreSQL.
Reescreva a consulta a seguir mantendo exatamente o mesmo resultado,
   porem de forma mais eficiente.
Responda APENAS com a consulta SQL reescrita, sem explicacao e sem
   markdown.
```

Nas fases seguintes, o *prompt* acrescenta blocos entre a instrução e a consulta original, separados por linha em branco: o esquema, no formato `tabela(coluna tipo, ...)`, e o número aproximado de linhas de cada tabela (C1 a C3), o catálogo de transformações (C2 e C3) e o plano do `EXPLAIN` da consulta original (C3). O catálogo enviado nas fases C2 e C3 é:

```
- Troque subconsultas correlacionadas por juncoes equivalentes quando
    o resultado for preservado.
- Prefira predicados sargaveis: nao aplique funcoes sobre a coluna
   filtrada; use faixas com >= e <.
- Remova DISTINCT quando as colunas projetadas ja forem unicas (por
   exemplo, uma chave primaria).
- Elimine subconsultas e juncoes que nao alteram o conjunto de
   resultados.
- Use CTEs (WITH) para evitar recomputar a mesma subconsulta varias
   vezes.
- Substitua agregacoes escalares calculadas por linha por uma unica
   agregacao com GROUP BY.
- Simplifique predicados e aplique os filtros o mais cedo possivel.
- Ao converter NOT IN em anti-juncao, atente para valores NULL, que
   alteram o resultado.
```

## APÊNDICE B – Exemplo completo de uma execução

O trecho abaixo mostra uma execução classificada como otimizada. A consulta é a `subconsulta_02`, reescrita pelo GPT-4o na fase C0. A consulta original usa uma subconsulta correlacionada com `count(*)` apenas para testar a existência de pedidos cancelados; a reescrita substitui a subconsulta por uma junção com `DISTINCT`.

```
-- original
SELECT c.id, c.nome
FROM cliente c
WHERE (SELECT count(*) FROM pedido p
       WHERE p.cliente_id = c.id AND p.status = 'cancelado') >= 1;

-- reescrita devolvida pelo modelo
SELECT DISTINCT c.id, c.nome
FROM cliente c
JOIN pedido p ON p.cliente_id = c.id
WHERE p.status = 'cancelado';
```

As duas consultas executaram e devolveram as mesmas 4.303 linhas; o custo estimado caiu de 196.761 para 1.237, um ganho de custo de 159, e a razão mediana de tempo foi de 6,28, acima de 1,15. A execução foi classificada como otimizada.